

ตอนที่ 4 — วันที่เราเลิกพึ่ง AI อย่างเดียว (25 สิงหาคม 2569)

ต่อจาก 2026-08-21(teach mk).md (ตอนที่ 3) คำศัพท์พื้นฐาน (token, LoRA, VRAM, visual token, grammar-constrained decoding) อธิบายไว้ในตอนที่ 1-3 แล้ว ตรงนี้จะอ้างถึงเฉยๆ

สรุปสั้นที่สุดก่อนอ่านยาว: วันนี้มาประชุมกับอาจารย์ได้มา 4 ข้อ ผมไปหาข้อมูลครบทั้ง 4 แล้วได้ผลจริง 3 เรื่อง:

- สร้างเครื่องมือใหม่ `tools/pattern_recognition.py` — มาร์คตำแหน่ง เสา/คาน/ฐานราก ลงบนภาพแบบก่อสร้างโดยไม่ใช้ AI เลย ทดสอบได้ฐานราก 14/14 และ 12/12 ในสองบ้าน
- เพิ่มความละเอียดภาพที่โมเดลเห็นขึ้น 32% โดยไม่ต้องจ่ายเพิ่มสักบาท — พบว่าที่ผ่านมา ภาพในชุดเทรน โดนย่อทุกใบ 100% ทั้งที่ไม่มีใครรู้ (บทที่ 2.3)
- พบว่า "section ได้ 0%" ส่วนใหญ่ไม่ใช่ความผิดของ AI แต่เป็นตัววัดที่วัดสิ่งที่เป็นไปไม่ได้ ต่อให้ AI สมบูรณ์แบบก็ทำได้ไม่เกิน 4% ในบ้านที่ใช้วัด (บทที่ 7)

ประเด็นสำคัญที่สุดของวันนี้ไม่ใช่เครื่องมือ แต่เป็นแนวคิด 2 อย่าง: (ก) ที่ผ่านมาระบายปัญหา ทุกอย่างด้วยการ "ทำให้ AI เก่งขึ้น" วันนี้เป็นครั้งแรกที่เรา แก้ด้วยการไม่ใช้ AI ในส่วนที่ไม่จำเป็นต้องใช้ และ (ข) ก่อนจะแก้เลขที่แยะ ต้องรู้ก่อนว่าเลขนั้นวัดอะไรอยู่

1. ย้อนดูก่อน — ปัญหาที่ยังแก้ไม่ได้จากคืนก่อน

1.1 t03 เทรนเสร็จแล้ว แต่ยังไม่ชนะ

เมื่อคืน (24-25 ส.ค.) เราเทรนโมเดลรอบที่ 3 เสร็จ ผลออกมาแบบนี้:

งาน (subtask)	t02 (รอบก่อน)	t03 (รอบล่าสุด)
schedule (ตารางเหล็ก)	0%	21.9% ✓ ดีขึ้น
plan_slab (ผังพื้น)	0%	20% ✓ ดีขึ้น
plan_footing (ผังฐานราก)	33.3%	33.3% เท่าเดิม
section (รูปตัด)	0%	0% เท่าเดิม
plan_beam (ผังคาน)	0%	0% ✗ ไม่ขยับเลย

บรรทัดสุดท้ายคือปัญหา เพราะ ผังคานคือเป้าหมายหลักของทั้งรอบ เราเทรนใหม่ทั้งรอบ เสียเงินไป ~\$5 เสียเวลา 5 ชั่วโมง เพื่อจะพบว่าเรื่องที่ตั้งใจแก้ที่สุด ไม่ขยับเลยแม้แต่นิดเดียว

1.2 "element recall" คืออะไร — คำที่ต้องเข้าใจก่อนอ่านต่อ

คำนี้จะโผล่ทั้งเอกสาร ต้องเข้าใจก่อน

สมมติหน้าแปลนคานหนึ่ง **มีคานจริง 15 ตัว** (นี่คือ "เฉลย" ที่คนนั่งอ่านแบบแล้วจذبไว้) เราให้ AI อ่านหน้าเดียวกัน AI ตอบมา 3 ตัว และถูกจริงแค่ 2 ตัว

- **recall (การเรียกคืน)** = ของจริงที่หาเจอ ÷ ของจริงทั้งหมด = $2 \div 15 = 13\%$
- แปลไทยตรงๆ ว่า **"เจอกี่เปอร์เซ็นต์ของที่ควรเจอ"**

recall 0% แปลว่า **AI หาไม่เจอเลยสักตัว** — ไม่ใช่หาเจอแต่อ่านค่าผิด แต่คือไม่เห็นมันเลย

1.3 ทำไมมันถึงแยขนาดนั้น

พอไปดูของจริงที่ AI ตอบ พบว่าในหน้าแปลนคานที่ยาก มันเขียนแบบนี้:

```
{
  "png": "20",
  "sheet_code": "S-03",
  "elements": []      ← วางเปล่า
}
```

มันเขียนแค่ข้อมูลหัวกระดาช แล้วปิดไฟล์เลย โดยไม่ใส่คานสักตัว — และที่แย่กว่านั้น ข้อมูลหัวกระดาชที่มันเขียนยังชี้ผิดบ้านด้วยซ้ำ (บอกว่าเป็นบ้าน 22 ทั้งที่กำลังอ่านบ้าน 08) เหมือนมันจำแบบฟอร์มจากตัวอย่างที่เคยเห็นมาแปะ โดยไม่ได้ดูภาพตรงหน้าจริงๆ

นี่ไม่ใช่ปัญหา "อ่านตัวเลขพลาด" แต่เป็นปัญหา **"มองไม่เห็นว่ามีของอยู่ตรงนั้น"** — และนี่คือเบาะแสสำคัญที่นำไปสู่งานวันนี้

2. ประชุมกับอาจารย์ — 4 ข้อ และคำตอบของแต่ละข้อ

มะขามกลับมาพร้อม 4 หัวข้อ ผมไปหาข้อมูลทั้งหมดแล้ว ขอตอบทีละข้อแบบตรงไปตรงมา **การรู้ว่าจะอะไรใช้ไม่ได้ก็มีค่าพอๆ กับรู้ว่าจะอะไรใช้ได้** เพราะประหยัดเงินและเวลา

ข้อสังเกตที่น่าสนใจก่อนอ่าน: ข้อ 3 ของอาจารย์ชี้ไปที่ตัวเลขที่ผิด (768 เป็นค่าของ พีเจอร์วีดีโอที่เราไม่ใช้) แต่คำถามนั้นพาไปเจอปัญหาจริงที่ใหญ่กว่า — ภาพโดนย่อทุกใบ โดยไม่มีใครรู้ คำถามผิดที่ถามถูกจุด ยังมีค่ากว่าคำตอบที่ไม่มีใครตรวจ

2.1 ข้อ 1 — "ทำ dictionary แปลไทยเป็นอังกฤษ" → มีของจริงรองรับ แต่ต้องทำให้ถูกวิธี

อาจารย์เห็นอะไร: คำสั่ง (prompt) ที่เราเขียนให้ AI เป็นภาษาอังกฤษหมด แต่ตัวแบบก่อสร้าง เป็นภาษาไทยหมด อาจารย์สงสัยว่าตรงรอยต่อนี้จะทำให้ AI สับสนไหม

สิ่งที่ผมไปเจอ: มีเทคนิคที่มีชื่อทางวิชาการจริง เรียกว่า **DIP (Dictionary Insertion Prompting)** — งานวิจัยปี 2024 วิธีคือ **แทรกคำแปลสั้นๆ ของศัพท์สำคัญเข้าไปในคำสั่ง** เช่น

คำศัพท์ที่จะเจอบนแบบ:

เสา = column, คาน = beam, ฐานราก = footing
เหล็กปลอก / ปล. = stirrup, ตอม่อ = pedestal

ไม่ใช่การแปลข้อความบนแบบ แต่เป็นการใส่พจนานุกรมย่อไว้ข้างๆ ให้ AI เปิดดูได้

คำเตือนสำคัญที่เจอจากอีกงานวิจัย: ถ้าแปลคำสั่งทั้งหมดเป็นอังกฤษ ความแม่นยำอาจ **ลดลง ~8%** เพราะภาษาต้นทางมีบริบทที่คำแปลทำลาย → ดังนั้นต้องทำแบบ "แทรกศัพท์" ไม่ใช่ "แปลทั้งหมด"

สถานะ: ยังไม่ได้ทำ รอถามอาจารย์ว่าอยากแก้ปัญหาข้อไหนกันแน่

2.2 ข้อ 2 — "เอา pattern recognition มาใช้ (cv2 template matching)" → ทำแล้ววันนี้

อันนี้คือเนื้อหาหลักของเอกสารนี้ ข้ามไปอ่านบทที่ 3 ได้เลย

2.3 ข้อ 3 — "max_frames 768 อยากเพิ่ม" → เพิ่มไปก็ไม่มีผลอะไรเลย

อันนี้ต้องอธิบายละเอียดหน่อย เพราะเป็นกรณีศึกษาที่ดีมาก

เลข 768 มีอยู่จริง ผมไปหาเจอในไฟล์ `processor_config.json` ของ t03:

```
"video_processor": {  
    "max_frames": 768,  
    "fps": 2  
}
```

← สังเกตคำนี้ให้ดี

แต่คำว่า video_processor คือกฎแฉ — `max_frames` แปลว่า "จำนวนเฟรมสูงสุดของวิดีโอ" โมเดล Qwen รับได้ทั้งภาพนิ่งและวิดีโอ ถ้าป้อนวิดีโอเข้าไป มันจะตัดวิดีโอเป็นภาพนิ่งทีละเฟรม และตัดได้มากที่สุด 768 เฟรม

แต่เราไม่เคยป้อนวิดีโอเลยสักครั้ง เราป้อนภาพแบบก่อสร้างทีละหน้า เป็นภาพนิ่งล้วน โค้ดของเราเขียนไว้ชัดเจนว่า

`{"type": "image", ...}` ไม่เคยมี `"video"` โผล่มาเลย

เปรียบเทียบให้เห็นภาพ: เหมือนรถเราวิ่งช้า แล้วมีคนแนะนำให้ "ปรับเกียร์ถอยหลังให้แรงขึ้น" — เกียร์ถอยหลังมีอยู่จริงในรถ ปรับได้จริง แต่เราไม่เคยใช้มันเลยตอนขับไปข้างหน้า ปรับไปก็เท่านั้น

แล้วปุ่มที่คุณเรื่องนี้จริงๆ คือปุ่มไหน? มี 2 ตัว ทั้งคู่อยู่ในไฟล์ `train_t03.py` :

ตัวแปร	ค่าปัจจุบัน	คืออะไร
<code>MAX_PIXELS</code>	5,120 × 1,024	ความละเอียดต่อ 1 ภาพ — ภาพถูกย่อแคไหนก่อนให้ AI ดู
<code>MAX_LENGTH</code>	32,768	ความยาวรวมทั้งหมด — คำสั่ง + ภาพ + คำตอบ รวมกันได้ยาวแคไหน

`MAX_PIXELS` คือตัวที่รอบ t01 เคยตั้งพลาด (ไปโดนย่อเหลือ 512px = ภาพเบลอมาก) แล้วทำให้ ทั้งรอบวัดผลผิด — บทเรียนข้อ 12 ใน `rule_of_tune.md`

แล้วเพิ่ม `MAX_LENGTH` ได้ไหม? ได้ในทางทฤษฎี — โมเดลตัวนี้รับได้ถึง **262,144** (เราใช้แค่ 32,768 = ใช้ไป 12% ของที่มันทำได้) **แต่ติดที่การ์ดจอ ไม่ได้ติดที่โมเดล** รอบที่แล้วเราต้องลดขนาด LoRA ลงครึ่งหนึ่งเพื่อให้ 32,768 พอดีการ์ด 95GB แล้วยังเกือบไม่พอ

2.3.1 พอไปวัดของจริง — เจอเรื่องที่ไม่มีใครรู้มาก่อน

มะขามสั่งให้ทำเรื่องนี้ต่อ ผมเลยเขียนเครื่องมือวัด (`measure_capacity.py`) แทนที่จะเดา คำนวณตรงจากขนาดไฟล์ ภาพ (patch 16 × merge 2 → 1 visual token = 32×32 = 1,024 พิกเซล)

ผลที่ได้ทำให้ตกใจ:

ขนาดภาพในชุดเทรน	จำนวน	ต้องใช้ token	cap เดิม 5,120
3309×2339	615 ใบ	7,558	เสีย 32%
3355×2347	176 ใบ	7,690	เสีย 33%
4631×3473	116 ใบ	15,707	เสีย 67%
3307×2339	65 ใบ	7,554	เสีย 32%

ภาพในชุดเทรน 973 จาก 973 ใบ = 100% โดนย่อทั้งหมด ไม่ใช่บางใบ — แม้แต่ภาพที่เล็กที่สุด ในชุด (3307×2339) ก็ยังเกิน cap

เราเคยบันทึกไว้ว่า "ภาพไม่โดนย่อเลย" ซึ่งมาจากการวัดตัวอย่างเดียวแล้วสรุปเหมารวม — **บทเรียนเดิมซ้ำรอย** (เหมือนข้อ 12 ใน `rule_of_tune.md` ที่รอบ t01 เคยพลาดมาแล้ว)

2.3.2 แล้วต้องเข้าการ์ดใหญ่ขึ้นไหม? — คำตอบคือ "ไม่ต้อง"

VRAM แบ่งเป็น 3 ส่วน แต่มีแค่ส่วนเดียวที่โตตามความละเอียดภาพ:

ส่วน	ขนาด	โตตามภาพใหม่
น้ำหนักโมเดล	74.0 GB	✗ นิ่ง
LoRA + gradient + optimizer	5.7 GB	✗ นิ่ง
activation	7.8 GB	✓ โดเชิงเส้น

79.7 GB แรกไม่ขยับเลย → เหลือที่ว่างให้เล่นได้ ~15 GB บนการ์ด 96GB เดิม

ตั้งเป็น	ภาพที่ยังโดนย่อ	peak VRAM	การ์ดเดิม 96GB
5,120 (เดิม)	100%	87.5 GB	✓ เหลือ 7.5
7,680 ← เลือกอันนี้	30%	90.9 GB	✓ เหลือ 4.0
16,384 (ไม่ย่อเลย)	0%	97.8 GB	✗ เกิน 2.8

ทำจริงแล้ว: แก่ MAX_PIXELS เป็น **7,680** และ MAX_LENGTH เป็น **47,104** ภาพ 3309×2339 จำนวน 615 ใบ (ส่วนใหญ่ของชุด) จะได้เทรนที่ความละเอียดเต็มเป็นครั้งแรก โดยไม่ต้องจ่ายเพิ่มสักบาท

ถ้าอยากไปถึง "ไม่ย่อเลย" ต้องเช่า **H200 140GB \$3.29/ชม. = \$17.45/รอบ** (การ์ดเดิม \$5.87) — เช็คราคาสดแล้ว แต่ **เครดิตเหลือ \$8.91** เทรนไม่จบรอบเดียว ต้องเติมเงินก่อน

⚠ ที่ยังต้องระวัง: ที่ว่างเหลือแคบลงจาก 7.5 → 4.0 GB ต้องรันทดสอบสั้น 5 step ยืนยันก่อนรันเต็มทุกครั้ง (รอบที่แล้วเคย OOM ตอนเหลือ 470 MB มาแล้ว)

สรุปข้อ 3: เลข 768 ที่อาจารย์เห็นเป็นของพีเจอร์วิดีโอที่เราไม่ใช้ แต่คำถามของอาจารย์ พาไปเจอของจริง — ปุ่มที่ควรปรับคือ MAX_PIXELS และมันถูกตั้งไว้เตี้ยเกินจริงมาตลอด คำถามถูก แม้ตัวเลขที่ชี้จะผิด

2.4 ข้อ 4 — "ใช้ Qwen เพิ่มขนาด B ให้มากๆ" → ✗ เราใช้ตัวใหญ่สุดที่โหลดได้อยู่แล้ว

"B" คืออะไร: ย่อจาก Billion (พันล้าน) = จำนวนพารามิเตอร์ พุดง่ายๆ คือ "ขนาดสมอง" ยิ่งเยอะยิ่งฉลาด แต่ยิ่งกินการ์ดจอ

เราใช้อยู่: Qwen3.6-35B-A3B = 35 พันล้าน

ตระกูล Qwen3.6 มีให้โหลดกี่ขนาด? ผมไปเช็คมาแล้ว มีแค่ 2 ตัว:

รุ่น	ขนาด	โหลดมาเทรนเองได้ไหม
Qwen3.6-27B	27B	ได้ (แต่เล็กกว่าที่เราใช้)
Qwen3.6-35B-A3B	35B	ได้ ← เราใช้ตัวนี้
Qwen3.6-Max-Preview	ใหญ่กว่า	ไม่ได้ — เป็นของปิด ใช้ได้ผ่านเว็บ/API เท่านั้น

คือเราอยู่บนยอดสุดแล้ว ไม่มีตัวใหญ่กว่านี้ให้โหลดในตระกูลนี้

ถ้าจะข้ามไปตระกูลอื่นที่ใหญ่กว่าจริง? มี — Qwen3-VL-235B-A22B (235 พันล้าน) แต่ตัวเลขค่าใช้จ่ายเป็นแบบนี้:

	ที่เราทำอยู่	ถ้าใช้ 235B
การ์ดจอที่ต้องใช้	1 ใบ (96GB)	8 ใบ (80GB ต่อใบ)
หน่วยความจำรวมตอนเทรน	~87GB	~770GB
ราคาต่อชั่วโมง	~\$1.1	หลายเท่าตัว
ความซับซ้อน	รันสคริปต์เดียว	ต้องเขียนใหม่ให้กระจายงานข้าม 8 การ์ด

สรุปข้อ 4: ไม่ใช่ "ปรับตัวเลขให้ใหญ่ขึ้น" แต่คือ "เปลี่ยนสเกลโครงการทั้งหมด" เป็นการตัดสินใจเรื่องงบ ไม่ใช่เรื่องเทคนิค

3. Template Matching คืออะไร — สอนตั้งแต่ศูนย์

ตอนนี้เข้าเนื้อหาหลัก อธิบายจากศูนย์ ไม่ต้องมีพื้นฐานอะไรมาก่อน

3.1 หลักการ: เกม "จับผิดภาพ" ที่คอมพิวเตอร์เล่นแทนเรา

ลองนึกภาพนี้:

- คุณตัดกระดาษเป็นช่องเล็กๆ ขนาดเท่ารูปฐานราก 1 อัน เรียกว่า **template (แม่แบบ)**
- คุณเอาช่องนั้นไปทาบบนแบบก่อสร้างแผ่นใหญ่ ที่มีย้ายบนสุดก่อน
- ให้คะแนนว่า "ตรงนี้เหมือนแม่แบบแค่ไหน" — ได้คะแนน 0 ถึง 1
- เลื่อนไปขวา 1 พิกเซล** ให้คะแนนใหม่
- ทำแบบนี้จนครบทั้งแผ่น ทีละพิกเซล

จบแล้วเราจะได้ "แผนที่คะแนน" ที่บอกว่าทุกตำแหน่งบนแบบเหมือนแม่แบบแค่ไหน ตรงไหนคะแนนสูง = ตรงนั้นมีของที่เราหา

คำสั่งที่ทำทั้งหมดนี้มีบรรทัดเดียว:

```
คะแนน = cv2.matchTemplate(ภาพใหญ่, แม่แบบ, cv2.TM_CCOEFF_NORMED)
```

cv2 คือชื่อไลบรารี OpenCV — เป็นเครื่องมือประมวลผลภาพที่ใช้กันทั่วโลกมา 20 ปี ฟรี

3.2 ทำไมมันถึงเร็วมาก (และไม่ต้องใช้การ์ดจอ)

แบบ 1 หน้าของเรามีขนาด 3309×2339 พิกเซล = **7.7 ล้านตำแหน่ง** ที่ต้องทาบและให้คะแนน ฟังดูเยอะมาก แต่เครื่องทำเสร็จใน **ไม่ถึง 1 วินาที** บน CPU ธรรมดา

เพราะมันเป็นแค่การคูณ-บวกเลข ไม่มีการ "คิด" อะไรเลย ต่างจาก AI ที่ต้องคำนวณผ่าน โครงข่ายประสาทหลายพันล้านตัว

นี่คือเหตุผลที่เครื่องมือนี้มีค่า: มันฟรี เร็ว และให้ผลเหมือนเดิมทุกครั้ง รันสิบลรอบได้คำตอบเดียวกันเป๊ะ ต่างจาก AI ที่ตอบไม่เหมือนกันสองครั้ง

3.3 6 วิธีให้คะแนน — และทำไมเราเลือกวิธีนี้

OpenCV ให้เลือก 6 สูตรในการให้คะแนนความเหมือน:

สูตร	อ่านผลยังไง
<code>TM_SQDIFF</code> , <code>TM_SQDIFF_NORMED</code>	ยิ่งน้อยยิ่งเหมือน (วัดความต่าง)
<code>TM_CCORR</code> , <code>TM_CCORR_NORMED</code>	ยิ่งมากยิ่งเหมือน
<code>TM_CCOEFF</code> , <code>TM_CCOEFF_NORMED</code>	ยิ่งมากยิ่งเหมือน ← เราใช้ตัวนี้

ทำไมเลือก `TM_CCOEFF_NORMED` :

- คำว่า **NORMED** (normalized = ปรับให้เป็นมาตรฐาน) แปลว่าคะแนนอยู่ในช่วง **0 ถึง 1** เสมอ ทำให้เราตั้งเกณฑ์ได้ เช่น "เอาเฉพาะที่เกิน 0.6" ถ้าไม่ normalize คะแนนจะเป็นเลข อะไรก็ไม่รู้ ตั้งเกณฑ์ไม่ได้
- คำว่า **CCOEFF** (correlation coefficient) คือสูตรที่**หักค่าความสว่างเฉลี่ยออกก่อนเทียบ** สำคัญมากกับงานเรา เพราะแบบเป็นภาพ**สแกน** บางหน้าสแกนติดเทาๆ บางหน้าขาวจัด สูตรนี้จะดูที่ "ลวดลาย" ไม่ใช่ "ความสว่าง" → หน้าที่สแกนเข้มก็ยังจับได้

3.4 threshold (เกณฑ์) — ปุ่มที่ต้องหมุนเอง

พอได้แผนที่คะแนนแล้ว เราต้องบอกว่า "คะแนนเท่าไรถึงนับว่าเจอ" นี่คือ **threshold**

- ตั้งสูงไป (เช่น 0.95)** → เจอน้อย ของจริงหลุดไปเยอะ
- ตั้งต่ำไป (เช่น 0.3)** → เจอเยอะ แต่จับตัวหนังสือ/ขอบตารางมาด้วย

ค่าที่เราใช้จริง (ได้จากการลองจริง ไม่ใช่เดา):

ประเภท	เกณฑ์	เหตุผล
ฐานราก	0.60	รูปทรงชัดเจนมาก ตั้ง 0.5-0.8 ก็ได้ 14/14 เท่ากัน
เสา	0.65	รูปเล็ก ต้องเข้มกว่าหนอยไม้จันทน์
คาน	0.70	เป็นเส้นเรียบๆ คล้ายเส้นอื่นเยอะ ต้องเข้มสุด

3.5 NMS — ปัญหาที่ทุกคนเจอ และวิธีแก้

ปัญหา: ถ้าฐานรากอยู่ที่พิกัด (1240, 766) พอเราเลื่อนแม่แบบไปที่ (1241, 766) — เลื่อนไป แค่ 1 พิกเซล — มันก็ยังไม่เหมือนอยู่ดี คะแนนยังสูง

ผลคือฐานราก 1 อัน ทำให้เกิดจุดคะแนนสูงเป็นร้อยจุดกระจกรอบๆ กัน ของจริงเรามีฐานราก 14 อัน แต่ระบบรายงาน 235 จุด

วิธีแก้ชื่อ NMS (Non-Maximum Suppression) แปลตรงตัวว่า "การกดทับตัวที่ไม่ใช่ที่สุด" หลักการง่ายมาก:

- เรียงทุกจุดจากคะแนนสูงสุดลงมา
- หยิบจุดคะแนนสูงสุดมาเก็บไว้ = "อันนี้ของจริงแน่"
- ลบทุกจุดที่อยู่ใกล้จุดนั้น (ใกล้กว่าครึ่งหนึ่งของขนาดแม่แบบ) — เพราะมันคือตัวเดียวกัน
- หยิบตัวถัดไปที่ยังเหลือ วนกลับข้อ 2

ผลจริงของเรา: 235 จุด → NMS → **14 จุด** ตรงกับของจริงเป๊ะ

3.6 ข้อจำกัดที่ต้องรู้ตั้งแต่แรก

จากงานวิจัยที่ผมไปอ่าน (SkeySpot, 2025 — งานตรวจจับสัญลักษณ์บนแบบไฟฟ้า) บอกไว้ชัด:

Template matching ฟังเมื่อ:

- สัญลักษณ์หมุน** — ถ้าแม่แบบเป็นแนวตั้งแต่ของจริงเอียง 30 องศา จะจับไม่ได้เลย
- สัญลักษณ์ขนาดต่างกัน** — แม่แบบใหญ่ 56px แต่ของจริง 40px ก็ไม่ตรง
- สัญลักษณ์วาดคนละแบบ** — คนละสำนักออกแบบวาดไม่เหมือนกัน

ข้อ 2 และ 3 เราเจอจริงวันนี้ทั้งคู่ (เล่าในบทที่ 5) งานวิจัยแนะนำให้ใช้ Deep Learning (YOLO) แทน — แต่ YOLO ต้องมีคนนั่งวาดกรอบสอน เป็นพันภาพก่อน ซึ่งเราไม่มีเวลา template matching จึงเป็นทางที่คุ้มกว่าสำหรับตอนนี้

4. สิ่งที่เราสร้างวันนี้ — เติบโตทีละส่วน

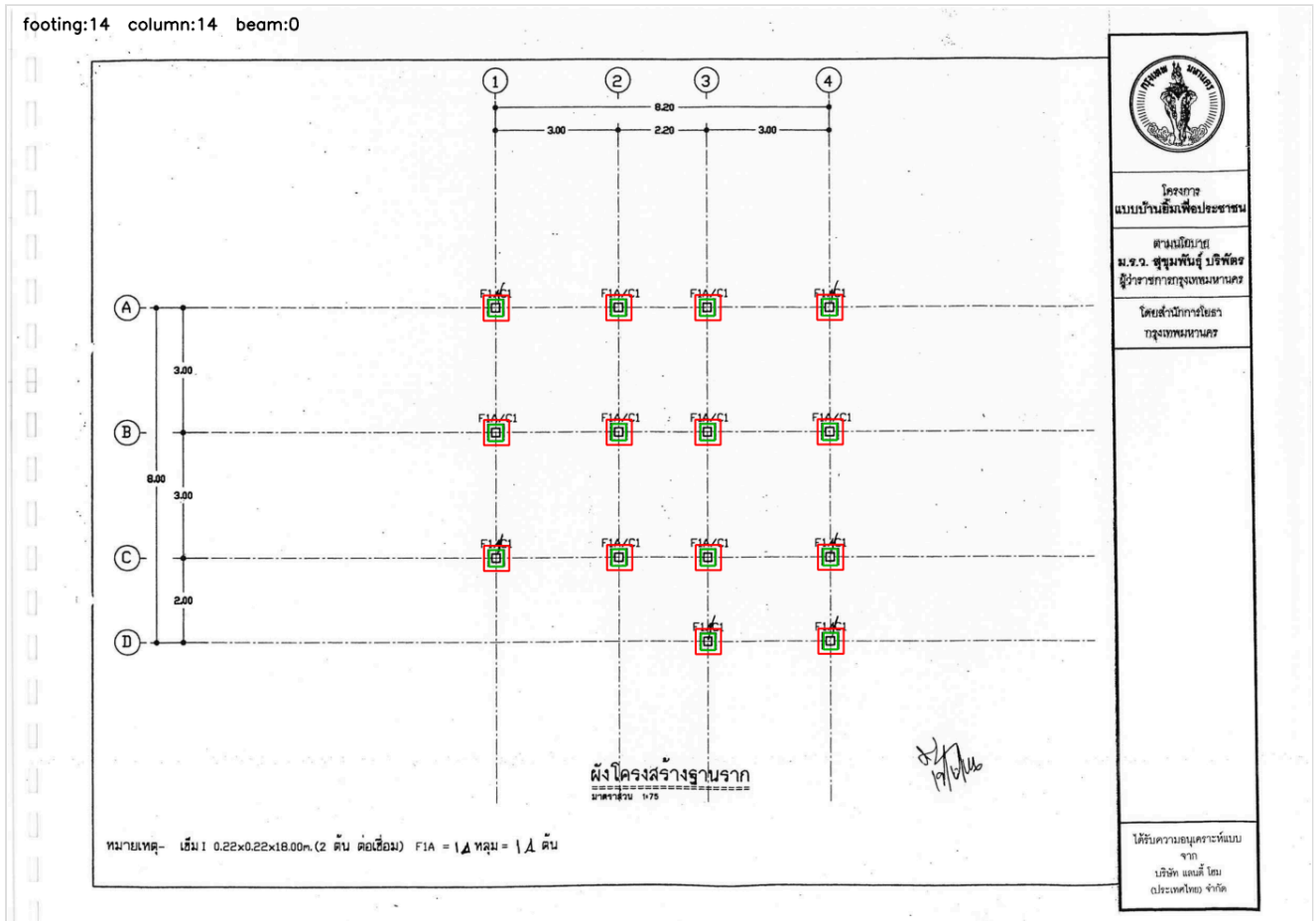
4.1 หน้าตาของเครื่องมือ

ไฟล์: `tools/pattern_recognition.py`

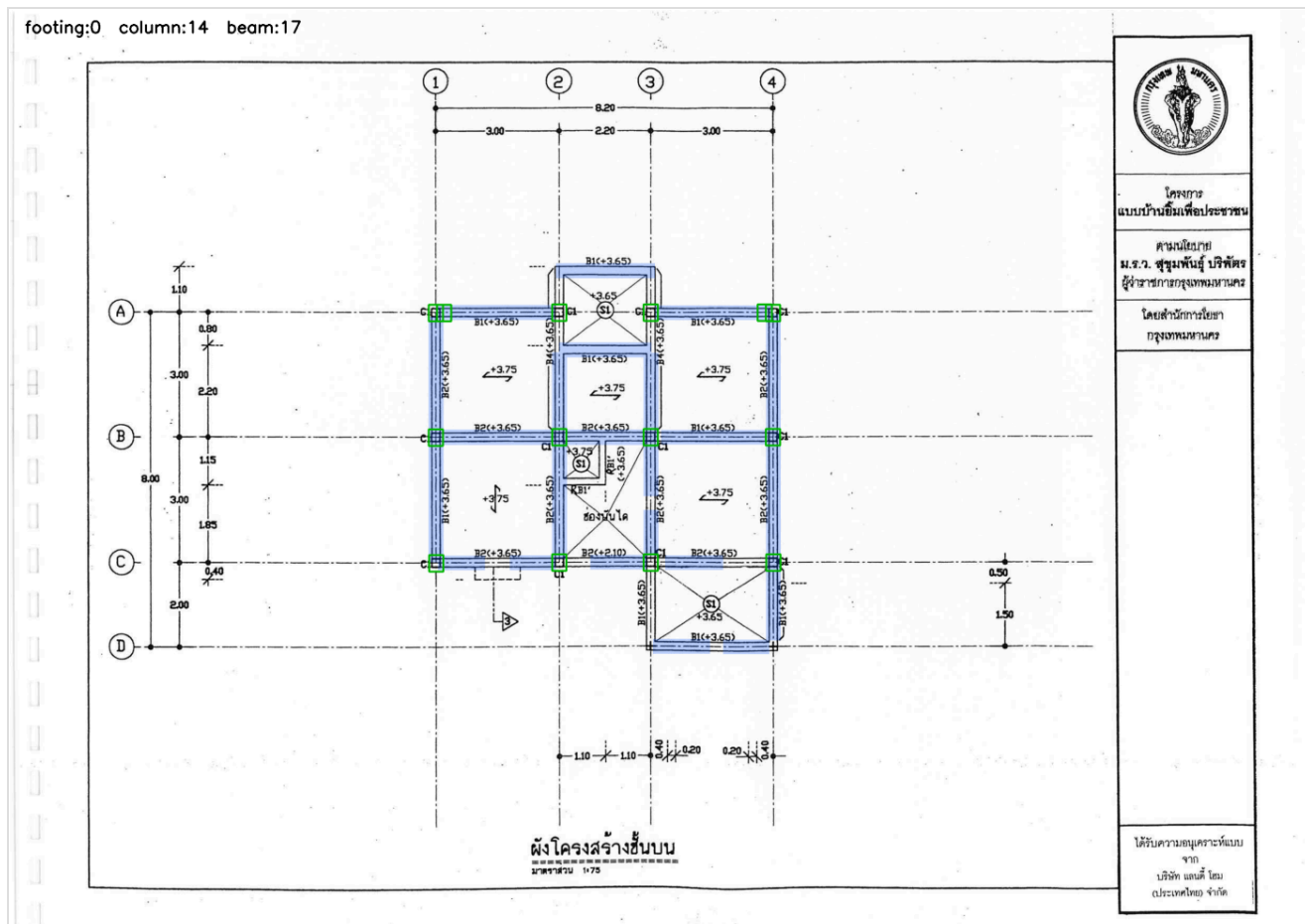

```
python tools/pattern_recognition.py "image/บ้านเล็ก_2ชั้น_17/..._หน้า14.png"
```

ได้ภาพใหม่ออกมาที่ tools/pattern_out/..._หน้า14_marked.png หน้าตาแบบนี้:

- **กรอบแดง** = ฐานราก
- **กรอบเขียว** = เสา
- **แถบน้ำเงินโปร่ง** = คาน
- มุมซ้ายบนมีตัวเลขนับให้



ผังฐานราก บ้าน 17 — ฐานราก 14 อัน (กรอบแดง) ตรงกับของจริงครบ ไม่มีจับม้วนออกผัง



ผังคานาชั้นบน บ้าน 17 — แถบนำเงินคลุมโครงคานา กรอบเขี้ยวคือเสาที่จุดตัด และไม่มีกรอบแดงเลย (ถูกต้อง เพราะ ผังคานาไม่มีฐานราก)

4.2 แม่แบบมาจากไหน — ตัดจากแบบจริง

แม่แบบ 4 ตัวเก็บไว้ที่ [tools/templates/](#) ทุกตัวตัดมาจากภาพแบบจริง ไม่ได้วาดขึ้นเอง:

ไฟล์	คืออะไร	ตัดมาจาก
tpl_footing.png	ไอคอนฐานราก F1A (สี่เหลี่ยมซ้อน)	บ้าน 17 หน้า 14
tpl_footing2.png	ไอคอนฐานราก F2 (พื้นผ้า + จุดเข็ม 2 จุด)	บ้าน 12 หน้า 20
tpl_footing3.png	ไอคอนฐานราก F1 ของบ้าน 12	บ้าน 12 หน้า 20
tpl_column.png	กล่องเสา C1	บ้าน 17 หน้า 16

4.3 คานาพิเศษ — เราไม่ได้ตัดจากภาพ แต่วาดขึ้นเอง

คานาเป็นข้อยกเว้น เพราะคานาไม่มีขนาดคงที่ — คานาช่วงสั้นยาว 300px คานาช่วงยาวยาว 900px ตัดมาเป็นแม่แบบไม่ได้ เพราะไม่รู้จะตัดยาวเท่าไร

ที่คงที่คือ "หน้าตัด" ของมัน — บนแปลน คานาถูกวาดเป็น เส้นดำ 2 เส้นขนานกัน (คือขอบสองข้างของคานา) เราจึงวาดแม่แบบขึ้นเองแบบนี้:

```
def beam_template(gap=21, thick=3, length=60):  
    tpl = ขาวล้วน ขนาด (33 x 60)  
    tpl[6:9, :] = ดำ # เส้นขอบบนของคาน  
    tpl[27:30, :] = ดำ # เส้นขอบล่างของคาน  
    return tpl
```

เลข gap=21 (ระยะห่าง 21 พิกเซล) **ไม่ได้เดา** — ผมไปวัดจากภาพจริง พบว่าขอบคานบน อยู่ที่แถว y=758-760 และขอบล่างที่ y=777-779 → ห่างกัน 21 พิกเซล

จากนั้น: หาแม่แบบสั้นๆ นี้ทั่วทั้งหน้า → ได้จุดกระจายเป็นแนวยาว → **ต่อจุดที่ติดกันให้เป็นแถบเดียว** → ทั้งแถบที่สั้นกว่า 100 พิกเซล (กันจับตัวหนังสือ)

⚠ ข้อควรระวังที่บันทึกไว้ในโค้ดแล้ว: เลข 21 นี้ผูกกับความละเอียดสแกนของชุดนี้ (3309×2339, มาตรฐาน 1:75) ถ้าเจอบ้านที่สแกนคนละความละเอียด ต้องปรับเอง ด้วยตัวเลือก `--beam-gap`

4.4 ผลลัพธ์จริง — ตรวจสอบด้วยตาทุกภาพแล้ว

หน้า	ฐานราก	เสา	คาน	ตรวจแล้วพบว่า
บ้าน 17 ผังฐานราก	14/14	14	0	ไม่จับมั่วสักจุด
บ้าน 17 ผังคานชั้นบน	0	14	17 แถบ	แถบน้ำเงินคลุมโครงคานเกือบครบ
บ้าน 12 ผังฐานราก	12/12	13	0	จับได้ทั้ง F1 และ F2
บ้าน 12 ผังคานชั้นสอง	0	13	19 แถบ	—

สังเกตช่อง "ฐานราก" ในหน้าผังคาน = **0** — นี่คือสิ่งที่ถูกต้อง เพราะผังคานไม่มีฐานราก ถ้ามันขึ้นเลขอื่นแปลว่าจับมั่ว (และตอนแรกมันจับมั่วจริง — อ่านบทที่ 5)

5. บั๊ก 2 ตัวที่เจอวันนี้ — และวิธีคิดตอนแก้

ส่วนนี้สำคัญกว่าตัวเครื่องมือ เพราะเป็นวิธีคิดที่ใช้ซ้ำได้กับทุกงาน

5.1 บั๊กที่ 1 — F1 กับ F2 ไม่ได้ต่างกันแค่ "ขนาด" แต่ต่างกันที่ "รูปทรง"

อาการ: พอลองบ้าน 12 (บ้านที่ 2) ระบบเจอฐานรากแค่ **6 จาก 12**

สมมติฐานแรกของผม (ผิด): "คงเป็นเพราะบ้านนี้วาดไอคอนขนาดต่างกัน" ผมเลยใส่ **multi-scale** เข้าไป คือย่อ/ขยายแม่แบบเป็น 6 ขนาด (0.7, 0.8, 0.9, 1.0, 1.1, 1.2 เท่า) แล้วหาทุกขนาด

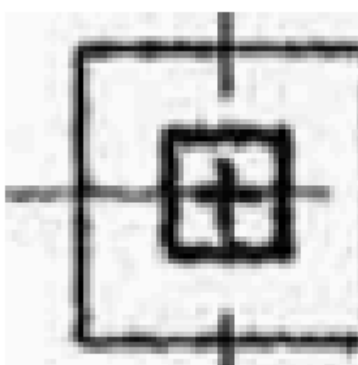
ผลคือยังได้ 6 เท่าเดิม → สมมติฐานผิด

จึงไปเปิดภาพดูจริง ตัดไอคอนสองอันมาดูข้างๆ กัน แล้วเห็นทันที:

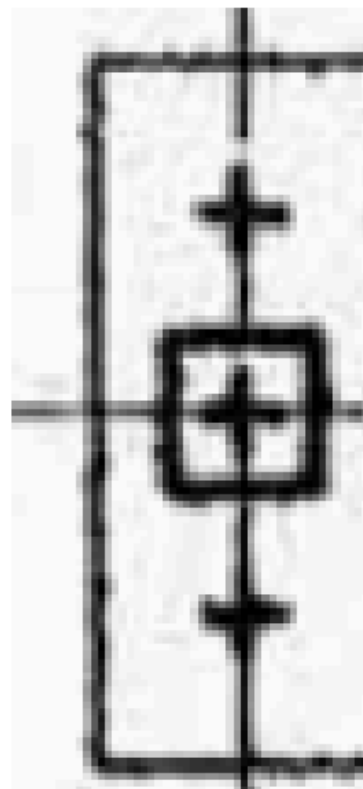
	F1	F2
รูปทรง	สี่เหลี่ยมจัตุรัสซ้อนกัน	สี่เหลี่ยมผืนผ้าแนวตั้ง
ข้างใน	กากบาท 1 อัน	กากบาท 1 อัน + จุดเพิ่ม 2 จุด บน-ล่าง



F1 (house 17)



F1 (house 12)



F2 (house 12)

แม่แบบจริงที่ใช้ — ซ้าย/กลางคือ F1 ของสองบ้าน (จัตุรัสซ้อน) ขวา คือ F2 (ผืนผ้า + จุดเพิ่มบน-ล่าง)

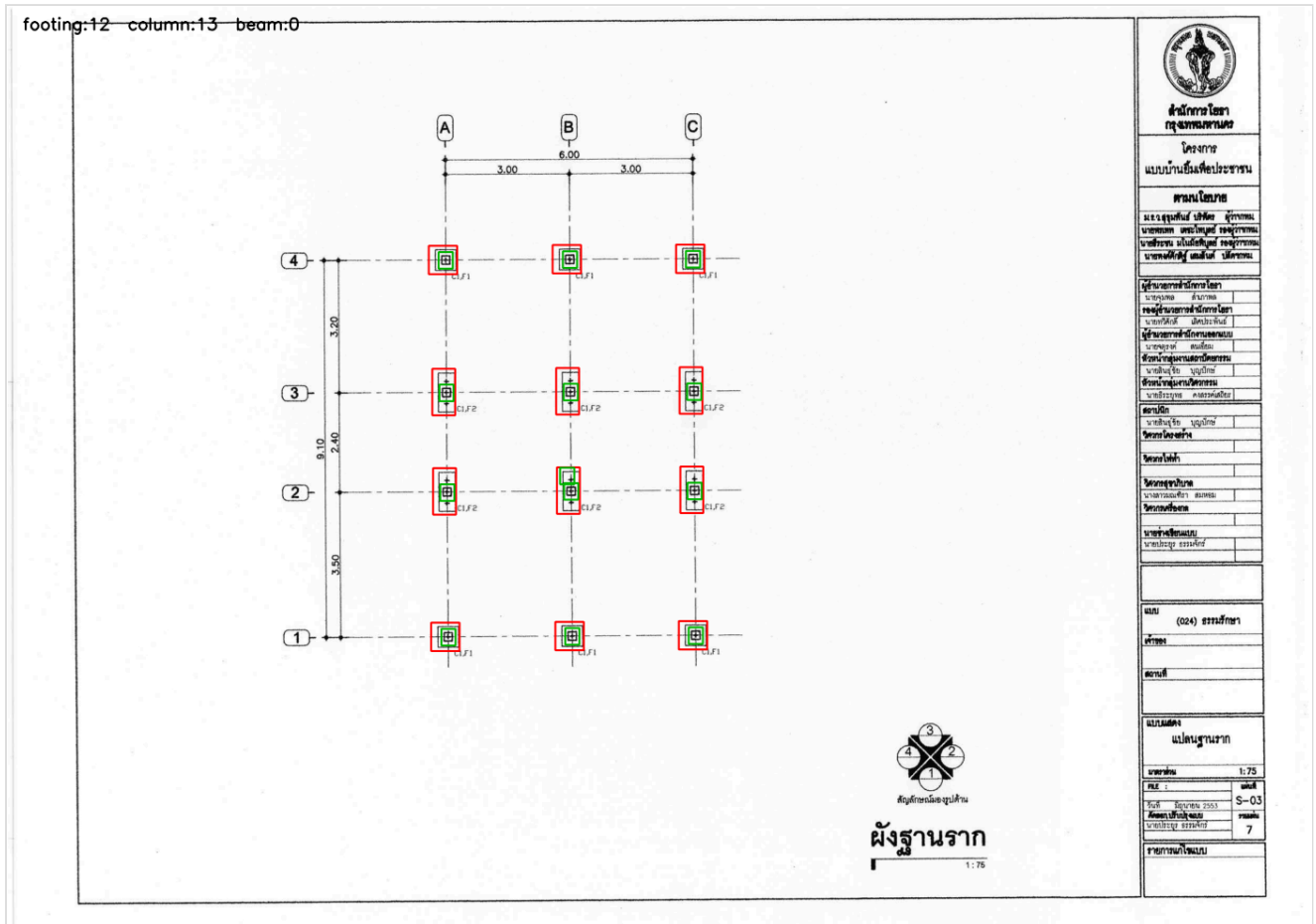
มันคือของคนละอย่างกัน ไม่ใช่ของอย่างเดียวกันคนละขนาด — F1 คือฐานรากเข็ม 1 ต้น F2 คือฐานรากเข็ม 2 ต้น ต่างกันโดยการออกแบบ ไม่ใช่ต่างกันโดยการวาด

วิธีแก้: **template bank (คลังแม่แบบ)** — แทนที่จะมีแม่แบบเดียว ให้เก็บหลายแบบไว้ในโฟลเดอร์ แล้วหาให้ครบทุกแบบ จากนั้นค่อยเอาผลมารวมกันแล้ว NMS ที่เดียว

```
def load_templates():
    # อ่านทุกไฟล์ที่ชื่อขึ้นต้นด้วย tpl_footing
    for p in sorted(TEMPLATE_DIR.glob("tpl_footing*.png")):
        ...
```

ข้อดีคือถ้าเจอบ้านที่วาดฐานรากแบบใหม่ แค่ตัดภาพวางเพิ่มในโฟลเดอร์ ไม่ต้องแก้โค้ดเลย

ผลหลังแก้: 12/12



บ้าน 12 หลังใส่คสลึงแม่แบบ — จับได้ทั้ง F1 (แถวบน/ล่าง) และ F2 (แถวกลาง) ครบ 12 อัน สังเกตว่ากรอบแดงของ F2 สูงกว่า เพราะไอคอนคนละครูปทรง

บทเรียน: สมมติฐานแรกที่ฟังดูสมเหตุสมผล (ขนาดต่างกัน) อาจผิดได้ วิธีพิสูจน์ที่ถูกคือเปิดของจริงมาดูด้วย

ตา ไม่ใช่เดาต่อแล้วเขียนโค้ดเพิ่ม — นี่คือนิยามเดียวกับข้อ 14 ใน `rule of tune.md`

5.2 บั๊กที่ 2 — ฐานรากโผล่ในผังคาน ทั้งที่ผังคานไม่มีฐานราก

อาการ: พอเอาแม่แบบ F2 (ผืนผ้าแนวตั้ง) ไปใช้ ปรากฏว่าในฝั่งคานของบ้าน 12 มั่นขึ้นกรอบแดง 3 อัน — ทั้งที่ฝั่งคานไม่มีฐานรากอยู่แล้ว

สาเหตุ: ลองนึกภาพ **คานแวนต์**วิ่งผ่านกล่องเสา — มันได้รูปเป็น "สี่เหลี่ยมผืนผ้าแวนต์ ที่มีอะไรอยู่ตรงกลาง" ซึ่งหน้าตาเหมือน **F2** มาก คอมพิวเตอร์เห็นแค่รูปทรง มันแยกไม่ออก

วิธีแก้ที่คิดไว้ออนแรก (ไม่ดี): เพิ่ม threshold ให้เข้มข้น → แต่จะทำให้ F2 จริงในผังฐานรากหลุดไปด้วย = ย้ายปัญหา ไม่ได้แก้

วิธีแก้ที่แท้จริง — ใช้ความรู้เรื่องแบบก่อสร้างมาช่วย:

ในผังฐานรากจริง ไม่มีคานให้ตรวจเจอเลย (เพราะเป็นผังคนละชั้นกัน) ดังนั้น: ถ้าจุดที่คิดว่าเป็นฐานราก
ต้นอยู่ในแถบคานที่ตรวจเจอ → ตัดทิ้ง

```
# ตัดฐานรากที่จุดกึ่งกลางตกอยู่ในแถบคาน
out["footing"] = [f for f in out["footing"]
                  if not any(อยู่ในแถบคาน(f, แถบ) for แถบ in beams)]
```

ทำไมกฎนี้ปลอดภัย 100%: เพราะในผังฐานรากจริง ระบบตรวจเจอคาน = 0 แถบ กฎนี้จึงไม่มีทางไปตัดฐานราก
จริงทิ้งได้เลย (ไม่มีแถบให้ตกลงไป)

ผลหลังแก้: ผังคานได้ฐานราก 0 ✔ และ ผังฐานรากยังได้ 12/12 เท่าเดิม ✔

บทเรียน: บางครั้งวิธีแก้ที่ดีที่สุดไม่ใช่การจูนตัวเลข แต่คือการใส่ความรู้เรื่องงานจริง เข้าไปเป็นกฎ — ความรู้
ที่ว่า "ผังฐานรากไม่มีคาน" เป็นความรู้ของวิศวกร ไม่ใช่ของโปรแกรมเมอร์

6. แล้วเครื่องมือนี้ช่วยเรื่อง recall ยังไง

กลับมาที่ปัญหาดังต้น: AI มองไม่เห็นคาน 15 ตัวในหน้าหนึ่ง

6.1 สิ่งที่เครื่องมือนี้ทำได้ / ทำไม่ได้

	template matching	AI (Qwen)
บอกว่า "ตรงนี้มีเสา"	✔ ทำได้	✔ ทำได้ (แต่พลาดบ่อย)
บอกว่า "เสาต้นนี้ชื่อ C1 หรือ C2"	✗ ทำไม่ได้	✔ ทำได้
บอกขนาด/จำนวนเหล็ก	✗ ทำไม่ได้	✔ ทำได้
ให้ผลเหมือนเดิมทุกครั้ง	✔ ได้	✗ ไม่ได้
ราคา	ฟรี	~\$1.1/ชม.

สองอย่างนี้ไม่ได้แข่งกัน แต่เติมเต็มกัน — เครื่องมือนี้เก่งเรื่อง "หาว่ามีอะไรตรงไหน" AI เก่งเรื่อง "อ่านว่ามันคืออะไร"

6.2 ทางที่เอาไปใช้ได้ — 3 ทาง เรียงจากปลอดภัยไปเสี่ยง

(ก) เป็นตัวช่วยคนตรวจงาน — ตอนมะขามนึ่งแก้ rawjson เปิดภาพที่มาร์คแล้วดูควบคู่ จะเห็นทันทีว่ามีคานตรง
ไหนที่ยังไม่ได้จด ไม่แต่ระบบอะไรเลย ปลอดภัยที่สุด

(ข) เป็นตัวเช็คว่า AI ตกหล่นไหม — รันเครื่องมือได้ 15 แถบคาน แต่ AI ตอบมา 3 ตัว → ระบบเตือนทันทีว่า "หน้านี้น่าจะพลาด" ทำให้เรารู้ว่าต้องยิ่งซ้ำ ไม่แตะข้อมูลเทรน ปลอดภัย

(ค) ป้อนตำแหน่งให้ AI ดูก่อนอ่าน — บอก AI ล่วงหน้าว่า "หน้านี้มีคานที่พิกัดเหล่านี้ ไปอ่านชื่อมันมา" ได้ผลแรงที่สุด แต่เป็นการแก้ที่ข้อมูลเทรน = เข้ากติกา rule_of_tune ข้อ 2 ต้องเตือนและขออนุญาตก่อนเสมอ — วันนี้ยังไม่ทำ

7. แล้ว section ที่ได้ 0% ละ — ใช้ pattern recognition ได้ไหม

มะขามถามต่อว่าเอาเครื่องมือวันนี้ไปช่วย section ได้ไหม ผมเริ่มลองทำเลย แล้วจับได้ว่าตัวเอง ทำผิดพลาด — ไปหาวิธีแก้ทั้งที่ยังไม่ได้วินิจฉัยว่าทำไมมันถึง 0% เลยถอยกลับมาวินิจฉัยก่อน แล้วคำตอบเปลี่ยนไปคนละเรื่อง

7.1 บทเรียนวิธีคิด: "เลขแยก" มี 2 แบบ ที่ต้องแยกให้ออก

- แบบ ก — โมเดลทำไม่ได้จริง → แก้ที่โมเดล/prompt/เครื่องมือช่วย
- แบบ ข — ตัววัดวัดสิ่งที่เป็นไปไม่ได้ → ต่อให้โมเดลสมบูรณ์แบบก็ยังไม่ได้ 0 แก้ที่โมเดลเท่าไรก็ไม่ขยับ

ถ้าแยกไม่ออกแล้วเดาผิด = เสียเงินเทรนทั้งรอบเพื่อแก้สิ่งที่ไม่ใช่ปัญหา

7.2 ไปดูของจริงว่า "0%" นับยังไง

โค้ดวัดผลใช้บรรทัดนี้:

```
hit = len(set(gt_ids) & set(pred_ids)) # เทียบสตริงตรงเป๊ะ
```

แล้วผมไปเปิดดูว่า element_id ในเฉลยของ section หน้าตาเป็นยังไงจริงๆ:

```
gate_front · door_lock_detail · SP_at_bathroom · purlin_rafter_connection  
FCO_detail · SP_unsupported_beam_edge · ห้องนอน · ระดับพื้นอาคาร
```

ชื่อพวกนี้ไม่ได้พิมพ์อยู่บนแบบ — เป็นชื่อที่คนจัดตั้งขึ้นเองเพื่อแยกรายละเอียดย่อย ไม่มีทางที่ AI (หรือคน) จะเดา SP_at_edge_beam ออกจากการมองรูป เพราะไม่มีตัวหนังสือมัน อยู่บนกระดาษเลยสักตัว

เปรียบเทียบ: เหมือนให้เด็กดูรูปหมาแล้วถามว่า "หมาตัวนี้ชื่ออะไร" — คำตอบคือ "ม้อบบี" ซึ่งเจ้าของตัวเองต่อให้เด็กสายตาดีแค่ไหนก็ตอบไม่ได้ นั่นไม่ใช่ความผิดของเด็ก

7.3 วัดออกมาเป็นตัวเลข — "เพดาน" ของแต่ละงาน

เขียนเครื่องมือ measure_id_ceiling.py แยกข้อ 2 พันธุ์แล้วนับ:

งาน	ชื่อทั้งหมด	อ่านได้จากแบบจริง	เพดานสูงสุด	t03 ทำได้	แปลว่า
plan_beam	2,484	2,175	88%	0%	โมเดลพังจริง
plan_slab	283	261	92%	20%	โมเดลพังจริง
plan_footing	462	345	75%	33%	พอมือชองให้ดีขึ้น
schedule	662	403	61%	22%	พอมือชองให้ดีขึ้น
section	2,436	902	37%	0%	ตัววัดพังครึ่งหนึ่ง
notes	272	5	2%	—	ตัววัดพังเกือบหมด

และเฉพาะบ้าน 08 ที่ใช้วัดผลจริง เพดาน **section = 1** จาก **23 = 4%** เพราะบ้าน 08 เป็นบ้านชั้นเดียวที่หน้า section แทบทั้งหมดเป็นรายละเอียด รั้ว/ประตู/ท่อน้ำ ไม่ใช่หน้าต่างคาน-เสา → **ต่อให้ AI สมบูรณ์แบบก็ทำได้ไม่เกิน 4%**

7.4 ข้อสรุป — 0% สองตัวนี้คนละโรคกัน

	plan_beam 0%	section 0%
เพดาน	88%	4% (บ้านที่วัด)
แปลว่า	โมเดลอ่านไม่ออกจริง	ตัววัดวัดสิ่งที่เป็นไปได้
pattern recognition ช่วยได้ไหม	✅ ใช่ นี่คือเป้าที่ถูก	❌ ไม่ได้ ไม่มีอะไรบนกระดาษให้จับ

ถ้าเอาสองตัวนี้ไปรวมกันแล้วสรุปว่า "โมเดลอ่านไม่เก่ง" = สรุปผิด plan_beam ต้องแก้ที่โมเดล ส่วน section ต้องแก้ที่ตัววัดก่อน มิฉะนั้นรอบหน้าจะไม่มีทางรู้เลย ว่าทำไปได้ผลหรือไม่

7.5 ลองแล้วจริงๆ ก่อนสรุป — บันทึกไว้กันคนอื่นทำซ้ำ

ไม่ได้สรุปลอยๆ ลองก่อน 3 วิธี:

วิธี	บ้าน 17	บ้าน 12	บ้าน 01
หาวงกลมล้อมชื่อ (B1)(B2) ด้วย HoughCircles	5/5 ✅ ไม่มั่วเลย	3/20 ❌	0/18 ❌

ได้ผลสวยมากกับบ้าน 17 แต่พังสนิทกับอีก 2 บ้าน เพราะ หน้า **section** ของแต่ละสำนักออกแบบ **วาดคนละแบบ** — บ้าน 17 ล้อมชื่อด้วยวงกลม, บ้าน 01 พิมพ์ "B1 1:20" ชัดเส้นใต้ ไม่มีวงกลมเลย ต่างจากผังโครงสร้าง (เสา/คาน/ฐานราก) ที่สัญลักษณ์นิ่งกว่ามาก — และนั่นคือเหตุผลที่ เครื่องมือวันนี้ได้ผลกับผังโครงสร้าง แต่ไม่ได้ผลกับ section

7.6 ทางแก้ที่เสนอ (ยังไม่ได้ทำ เพราะแต่ละเจเลย = ต้องขออนุญาตก่อน)

1. **แยกคอลัมน์ตัววัด** — รายงานคะแนนเฉพาะชื่อที่พิมพ์บนแบบ แยกจากชื่อที่คนตัวเอง (แก้แค่สคริปต์วัด ไม่แต่ละเจเลย = ปลอดภัยสุด ทำได้ทันทีถ้าสั่ง)

- 2. ล้างช่องว่าง/ตัวพิมพ์ใหญ่เล็กก่อนเทียบ — กัน "B1 " ไม่ตรงกับ "B1"
- 3. เปลี่ยนกติกการตั้งชื่อ element_id ของ section (แต่ละเฉลย = งานใหญ่ ต้องขออนุญาต)

8. สิ่งที่ยังไม่รู้ / ยังไม่ได้ทำ (ต้องพูดตรงๆ)

ตามกติกาของโปรเจกต์นี้ ต้องบอกให้ครบว่าจะอะไรที่ยังไม่ได้พิสูจน์:

- 1. ทดสอบแค่ 2 บ้าน และเป็นซีรีส์เดียวกัน (แบบบ้านยืมเพื่อประชาชน ของ กทม.) บ้านจากสำนักงาน ออกแบบอื่นอาจวาดสัญลักษณ์ไม่เหมือนเลย ยังไม่รู้ว่าจะฟังหรือไม่
- 2. ยังไม่เคยลองกับหน้าที่ไม่ใช่ผังโครงสร้าง — หน้าปก, รูปตัด, ตาราง BOQ ถ้ามันไปมาร์คมันหน้าพวกนี้ คนที่เอาไปใช้จะโดนหลอก อันนี้ควรทดสอบก่อนใช้จริง
- 3. มีจุดซ้ำเล็กน้อย — บ้าน 12 ผังฐานรากชั้นเสา 13 อัน ทั้งที่มี 12 (มีกรอบซ้อนกัน 1 จุด) ไม่ได้แก้ เพราะ กระทบภาพรวมน้อย
- 4. ยังไม่ได้ทำพื้น (S1) — มะขามสังขอบเขตแค่ เสา/คาน/ฐานราก จึงหยุดตามนั้น
- 5. ยังไม่ได้เอาไปเชื่อมกับอะไรทั้งสิ้น — เป็นเครื่องมือเดี่ยวๆ ที่ออกภาพอย่างเดียว ไม่แตะ rawjson ไม่แตะ prompt ไม่แตะข้อมูลเทรน

9. คำศัพท์วันนี้

คำ	แปลง่ายๆ
template	แม่แบบ — ภาพเล็กที่เราเอาไปทาบหา
template matching	การเลื่อนแม่แบบทาบทั่วภาพใหญ่แล้วให้คะแนนความเหมือน
threshold	เกณฑ์ — คะแนนขั้นต่ำที่จะนับว่า "เจอ"
NMS	การลบจุดที่ซ้อนกัน ให้เหลือตัวแทน 1 จุดต่อ 1 ของ
multi-scale	หาหลายขนาด — ย่อ/ขยายแม่แบบแล้วหาทุกขนาด
false positive	จับมั่ว — บอกว่าเจอ ทั้งที่ไม่มีของอยู่ตรงนั้น
recall	เจอกี่ % ของที่ควรเจอ
CPU / GPU	ซีพียู = สมองคอมทั่วไป (งานนี้ใช้แค่นี้) / จีพียู = การ์ดจอ (ต้องเช่า, แพง)
B (ในชื่อโมเดล)	พันล้านพารามิเตอร์ = ขนาดสมองของ AI
max_frames	จำนวนเฟรมสูงสุดของวิดีโอ (เราไม่ใช้)
MAX_LENGTH	ความยาวรวมที่ AI รับได้ต่อครั้ง (อันนี้เราใช้จริง)

10. ลองเองได้เลย — คำสั่งทั้งหมด

เปิด terminal ที่โฟลเดอร์ `Training` แล้วพิมพ์:

```
# 1. ทดสอบว่าเครื่องมือทำงานถูก (จะเห็นว่าได้ฐานราก 14 อันจริงไหม)
python tools/pattern_recognition.py --demo

# 2. รันกับหน้าไหนก็ได้
python tools/pattern_recognition.py "image/บ้าน_เล็ก_2ชั้น_17/บ้าน_เล็ก_2ชั้น_17_หน้า14.png"

# 3. ถ้าเจอบ้านที่สแกนคนละความละเอียด แล้วคานจับไม่ได้ ลองปรับระยะห่างขอบคาน
python tools/pattern_recognition.py "ภาพ.png" --beam-gap 30
```

ภาพผลลัพธ์อยู่ที่ `tools/pattern_out/` เปิดดูได้เลย

ถ้าเจอบ้านที่วาดฐานรากคนละแบบ วิธีเพิ่มแม่แบบเอง:

1. เปิดภาพ ครอบเฉพาะไอคอนฐานรากมา 1 อัน (เพื่อขอบนิดหน่อย)
2. เซฟเป็น `tools/templates/tpl_footing4.png` (เลขต่อจากที่มีอยู่)
3. รันใหม่ — โปรแกรมจะอ่านไฟล์ใหม่เองอัตโนมัติ ไม่ต้องแก้โค้ด

11. สรุปแบบสั้นที่สุด

1. **t03 ดีขึ้นหลายเรื่อง แต่ฝั่งคานยัง 0%** — ปัญหาคือ AI มองไม่เห็นของ ไม่ใช่อ่านผิด
 2. **ข้อ 2 (pattern recognition) ทำเสร็จแล้ววันนี้** — มาร์ค เสา/คาน/ฐานราก ได้จริง ฟรี เร็ว ไม่ใช่การ์ดจอ ไม่ใช่ AI
 3. **ข้อ 3 (max_frames) ชี้ตัวเลขผิด แต่พาไปเจอของจริง** — ภาพในชุดเทรนโดนย่อ **ทุกใบ 100%** แก่ แล้ว ได้ความละเอียดเพิ่ม **32% โดยไม่ต้องจ่ายเพิ่ม** (ไม่ต้องเข้าการ์ดใหญ่อย่างที่คิดตอนแรก)
 4. **ข้อ 4 (เพิ่มขนาด B) ใช้ไม่ได้** — เราอยู่บนตัวใหญ่สุดที่โหลดมาเทรนเองได้แล้ว
 5. **ข้อ 1 (dictionary) มีงานวิจัยรองรับจริง** แต่ต้องทำแบบ "แทรกศัพท์" ไม่ใช่ "แปลทั้งชุด" — ยังไม่ได้ทำ รอคอยกับอาจารย์
 6. **section 0% ไม่ใช่ความผิดของ AI** — ตัววัดใช้ชื่อที่คนตัวเอง (`gate_front` , `SP_at_bathroom`) ซึ่งไม่มีอยู่บนกระดาษ **เพดานสูงสุดของบ้านที่ใช้วัดคือ 4%** ต้องแก้ตัววัดก่อน ไม่ใช่แก้โมเดล
 7. **แนวคิดใหม่ที่ได้ 2 อย่าง:** (ก) ไม่ใช่ทุกปัญหาต้องแก้ด้วยการทำให้ AI เก่งขึ้น บางอย่างคณิตศาสตร์ธรรมดา ทำได้ดีกว่า ถูกกว่า เชื่อถือได้มากกว่า (ข) **ก่อนแก้เลขที่แป๊ะ ต้องรู้ก่อนว่าเลขนั้นวัดอะไร** ไม่งั้นอาจเสียเงิน ทั้งรอบ ไปแก้สิ่งที่ไม่ใช่ปัญหา
-

12. ภาคผนวก — ลองหาโมเดล AI ตัวอื่นนอกจาก Qwen

คืนเดียวกันนี้ มะขามสั่งลองหาโมเดล vision ขนาด 100-200B ที่ไม่ใช่ตระกูล Qwen (ทั้งชื่อแบรนด์และ "สมอง" ข้างใน) มาเทียบดู เช็คไป 8 ตัว

12.1 ตัวที่เก่งแต่ใช้ไม่ได้ — Pixtral-Large

- **ขนาด ~124B** (Mistral AI) — เก่งด้านอ่านเอกสาร/ตาราง/กราฟ และรองรับข้อความยาวมาก
- **มีระบบนับพิกเซล** คล้าย Qwen ที่เราใช้อยู่ — ไม่บังคับย่อภาพเป็นสี่เหลี่ยมตายตัว แต่หันเป็น patch ตามขนาดจริงของภาพ (ขนาด/สัดส่วนต่างกัน ใช้จำนวน patch ต่างกัน)
- **แต่ใช้ไม่ได้เลย** — ไลเซนส์เป็น **Mistral Research License** ห้ามใช้เชิงพาณิชย์ เว้นแต่ซื้อไลเซนส์แยกกับ Mistral เก่งแค่ไหนก็ถูกรอตั้งแต่ข้อนี้ข้อเดียว

12.2 ทำไมตัวอื่นก็ถูกรอเหมือนกัน

ตัว	ถูกรอเพราะ
InternVL3-78B / InternVL3.5-241B	แบรนด์อื่น แต่ข้างในใช้ Qwen2.5/Qwen3 เป็นสมอง — นับเป็น Qwen อยู่ดี
GLM-4.6V	ไลเซนส์โอเค แต่มีบั๊กเทรนแบบ LoRA ไม่ได้ (ยังไม่มีคนแก้)
Molmo-72B, NVLM-D-72B	ยังไม่มีเครื่องมือเทรนไหนยืนยันว่าใช้ได้จริง

เหลือรอดแค่ 2 ตัว: GLM-4.6V (ติดบั๊ก) กับ **Llama-4-Scout** (Meta, ใช้ฟรีถ้าผู้ใช้ < 700 ล้านคน/เดือน, เทรนได้จริง) — แต่ทั้งคู่ยังไม่มีใครทดสอบว่าอ่านภาษาไทยได้ดีแค่ไหน

12.3 อัปเดต — เช็คซ้ำอีก 2 ตัวที่เหลือค้าง

- **NVLM-D-72B (NVIDIA)** — ตัดขาดจริง ไลเซนส์ **CC-BY-NC-4.0** ห้ามใช้เชิงพาณิชย์เต็มๆ ห้ามใช้กับ STECON ตั้งแต่แรก ไม่ต้องพูดถึง framework เลย และ vLLM เองก็ยังไม่รันไม่ได้ (NVIDIA ยอมรับเองในกระทู้ HF)
- **ERNIE-4.5-VL-28B-A3B (Baidu)** — แก่ข้อมูลเดิม: จริงๆ มี **Unsloth notebook** ใช้ได้เลย (ก่อนหน้านี้เข้าใจผิดว่าต้องใช้ ERNIEKit เท่านั้น) แต่ตัวนี้แค่ **28B** เล็กกว่าเกณฑ์ 122B±50 ที่ขอ
- **เจอเพิ่ม:** OmniDocBench (leaderboard วัดผลจริงที่ยัง maintain อยู่) มี **InternVL3-78B** ได้ **83.76 คะแนน** เป็นตัวเดียวจาก 10 ตัวที่เช็คทั้งหมดที่ขึ้น leaderboard จริง — เพิ่มความน่าเชื่อถือ ให้ InternVL3-78B เล็กน้อย (แม้จะยังต้องเปลี่ยน framework จาก Unsloth ไป LLaMA-Factory และยั้มนับเป็น Qwen backbone อยู่ดี)

12.4 ตามหาตัวแทน NVLM-D-72B ต่อ — ยังไม่เจอตัวที่แทนได้เป๊ะ

มะขามถูกใจ NVLM-D-72B (ทั้งที่ถูกรอ) เลยขอให้หาต่อว่ามีตัวไหนคล้ายกันบ้าง — เจอ 3 ตัวใหม่ แต่ยัง**ไม่มีตัวไหนแทนได้ตรงสเปก** (72B + commercial + เทรนง่าย):

ตัว	ขนาด	ไลเซนส์	สรุป
Nemotron 3 Nano Omni (NVIDIA, เม.ย. 69)	30B (active 3B)	ใช้เชิงพาณิชย์ได้จริง 	ทายาทจิตวิญญาณของ NVLM แต่ เล็กกว่าที่ขอมาก license สะอาดสุดในบรรดาที่เข้คมาทั้งหมด
GLM-5V-Turbo (Z.ai, เม.ย. 69)	ไม่เปิดเผย	ปิด — API เท่านั้น	ดาวนโหลดมาเทรนเองไม่ได้เลย ดกรอบทันที
MiniMax M3	428B รวม / 22B active	มีเงื่อนไขซ่อน	ฟรีถ้าไม่ใช่เชิงพาณิชย์ แต่ถ้าเอาไป fine-tune ใช้งานจริงนับเป็น commercial ต้องขอไลเซนส์แยก — คล้ายดักของ Pixtral-Large ยังไม่ตัดขาด 100% ต้องอ่าน terms ละเอียดกว่านี้

สรุปภาพรวมข้อ 12: วงการตอนนี้แบ่งเป็น 2 ค่าย — **ค่ายเปิดจริง** (Qwen/DeepSeek/Gemma) แทบไม่มีตัว vision ขนาดกลางเลย กับ **ค่ายที่ vision เก่ง** (Mistral/MiniMax/NVIDIA) แต่เกือบทุกเจ้าผูกไลเซนส์แบบมีเงื่อนไขซ่อน ยังไม่เจอตัวที่ครบทั้ง 3 อย่าง (ขนาด+license สะอาด+เทรนได้จริง)

เอกสารนี้เขียนวันที่ 25 สิงหาคม 2569 — บันทึกการทำงานแบบละเอียดอยู่ที่ `2026-08-25.md` โค้ดจริงอยู่ที่ `tools/pattern_recognition.py`